VSTO & REFLECTION BROWNBAG

Created in Office 12 build 4518.1014

Background

- VSTO stands for Visual Studio Tools for Office
- Enables developers to create managed code solutions in Office applications
- ORCAS will provide VSTO support for other Office apps

VSTO

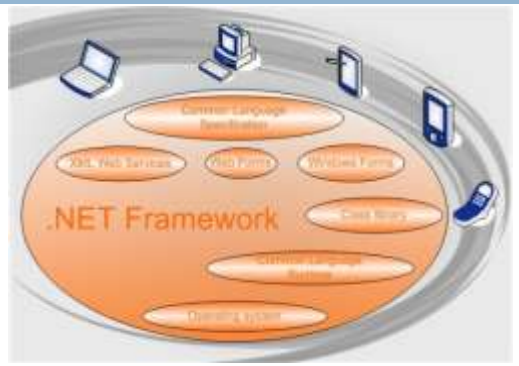
Visual Studio Tools for Office



What's in Orcas?

- Application level add-ins
- Ribbon designer
- Custom Task Pane support

Requirements



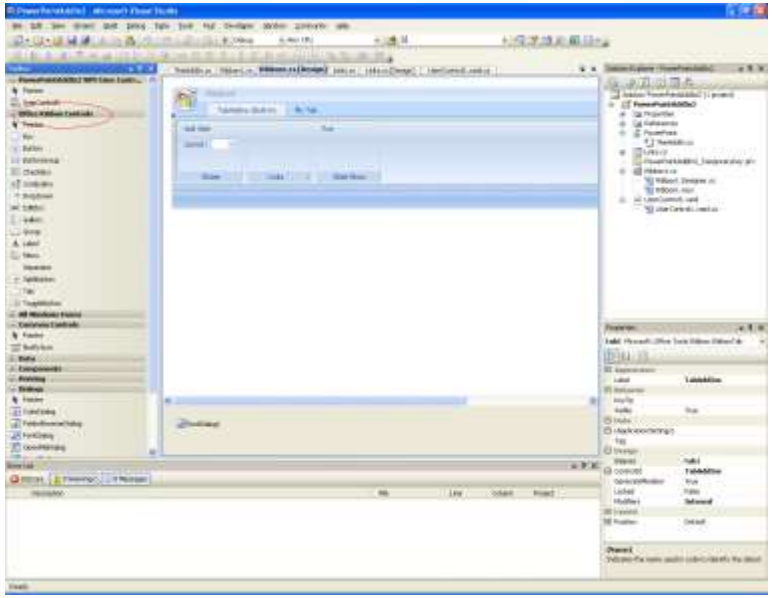
.NET Framework 3.5

Office 2003 or 2007

Visual Studio “Orcas” Beta1

Demo

- Creating a project
- Using the custom task pane
- Using the Ribbon Designer
- Deploying a project
- Sample Solutions



Resources

- Installation Instructions: http://office/14/teams/powerpoint/test/Tools/Tools%20Wiki/VSTO%20InstallationInstructions.aspx

Reflection

Reflection

- **Definition** *(from Wikipedia)* - Reflection is an activity in computation that allows an object to have information about its own structure and reason about its own computation.
- What does this really mean?
  - Late binding – Load dlls and libraries that are unknown at compile time
  - Extracting metadata from classes – Read attributes, properties, method signatures, etc.
  - Self-modifying code – Can generate MSIL code. Used in regex implementations . *not covered in this brownbag

Loading an Assembly

- We can load an assembly at runtime:
  - System.Reflection.Assembly assm = Assembly.LoadFrom(@“c:\myDLL.dll”);
- With this assembly object we can get the version number, types it contains, and more.
  - Get the version Number
    - Assm.GetName().Version.Major
    - Assm.GetName().Version.Minor
  - Get information about classes inside the assembly
    - Type[] types = assm.GetTypes();

Getting types

- The class Type is the workhorse of reflection. Once we get the type information we can instantiate objects, get attributes, access properties, and call methods. Essentially anything .NET can do with an object, you can do with the Type class.
- There are many ways to get a type.
  - From the assembly: assm.GetType("TypeName");
  - If you already have an object: o.GetType();
  - With the keyword typeof(PowerPoint.Shape)

GetType() and System.__ComObject

- Beware!, getType() won't work for ComObjects. Link to KB article
- Unfortunately, all our PowerPoint classes are of this type
- Suggested workaround: attempt casting the object as different types until it is non null

```
O = PPT.Selection();
if( (o as PPT.Shape ) != null){ \\object is a shape }
else if( (o as PPT.Slide ) != null){ \\object is a slide }
…
```

Getting Information about types

- The Type class information about the methods, properties, constructors, and more for that type
- To get all the methods, given a type t:
  - MethodInfo[] mInfos = t.GetMethods();
- To get all the properties
  - PropertyInfo[] pInfos = t.GetProperties();
- To get all the constructors
  - ConstructorInfo[] cInfos = t.GetConstructors();

Instantiating a Type

- By invoking the ConstructorInfo we can instantiate a type. The method accepts an array of objects for passing arguments to the constructor.
  - Object o = cInfo.Invoke(new object {“foo”,23});
- The Activator class provides a shortcut to instantiating types.
  - Object o = Activator.CreateInstance(t,new object[]{“foo”,23});

Invoking Methods and Properties

Invoking a method using a methodinfo, given a previously instantiated object, o.	• MethodInfo mInfo = t.GetMethod("MethodName"); • minfo.Invoke(o, new object[]{});
Invoking a property	• PropertyInfo pInfo = p.GetProperty("PropName");
Invoke the Get property. The second value is optional and is only used for indexing.	• pInfo.GetValue(o,null);
Invoke the Set property. Again, we need a null for the optional indexer.	• Pinfo.SetValue(o, "valueToSet", null);

Demo

- Create a console application that recursively prints out the property names and values for a shape in PowerPoint

Resources

- MSDN Reflection Overview: http://msdn2.microsoft.com/en-us/library/f7ykdhsy.aspx

Q&A